Carleton Smart Course Scheduler

SYSC 4907 Capstone Project

Helping students check prerequisites and build conflict-free schedules

January 2026

The Problem

Course registration at Carleton is a pain

Checking Prerequisites

- Open your audit, open the calendar, go back and forth
- “Wait, do I have SYSC 3303 or not?”
- Takes a while if you’re picking 5-6 courses

Info is Everywhere

- Carleton Central for one thing
- Course calendar for another
- Department sites for elective lists

Building a Schedule

- No way to check conflicts automatically
- Try a section, doesn’t work, try another
- Hope you don’t mess up LEC/LAB/TUT

Actually Registering

- Hunt down CRNs one by one
- Copy them into Central
- Pray nothing conflicts

What We Built

Carleton Smart Course Scheduler

Feature	What it does
Upload your audit	We parse it and know what you've taken
Search any course	Instantly see if you can take it or what's missing
Generate schedules	Get every possible conflict-free timetable
Copy CRNs	One click, paste into Central, done

Automates the tedious parts of course registration

Design Principle: User-Centered Design

Making It Simple — Before vs After

The Old Way

"Can I take SYSC 4001?"
⟶ Open audit in one tab
⟶ Open calendar in another
⟶ Find the prerequisites
⟶ Check each one against your audit
⟶ Figure out the AND/OR logic

Repeat for every course

With Our App

"Can I take SYSC 4001?"
⟶ Type "SYSC 4001"
⟶ See "You're good" or "Need SYSC 3303"

Instant feedback

Upload once, and the app remembers what you've completed

Design Principle: Constraint Satisfaction

Too many combinations to check by hand

The Problem

- Pick 5 courses
- Each course may have multiple sections
- Each section may have LEC, LAB, TUT components
- All combinations need to be checked for time conflicts

Doing this manually is tedious and error-prone.

How We Do It: Backtracking

```
generateSchedules(courses, current):
  if all courses scheduled:
    save it, return
  for section in nextCourse:
    if no conflict:
      add section, recurse, remove
    else: skip this branch
```

If something conflicts, stop early and try the next option. This saves time early.

Design Principle: Separation of Concerns

Three layers, each does one job

Frontend — React + TypeScript

What you see and interact with

↓ talks to ↓

Backend — Express + Node.js

Handles requests, runs the logic, talks to the database

↓ queries ↓

Database — PostgreSQL

Stores courses, sections, and prerequisite rules

Design Principle: Data Modeling

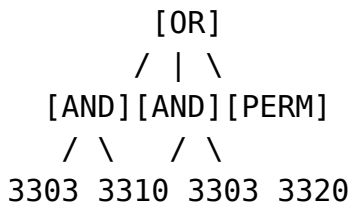
Prerequisites aren't just a list — they're logic

The Problem

Prerequisites have AND/OR conditions:

> SYSC 4001 needs:
> (SYSC 3303 AND SYSC 3310)
> OR (SYSC 3303 AND SYSC 3320)
> OR (Permission of instructor)

Our Solution: A Tree



Walk the tree, check each node. If any OR branch passes, you're good.

Design Principle: Performance Optimization

Don’t hammer the database

Without Optimization

User types “SYSC4001”
⟶ 8 keystrokes = 8 API calls
⟶ 8 database queries
Multiply by many concurrent users…

What We Do Instead

Debounce — wait 300ms after typing stops
Cache — remember results for 5 min
Min length — don’t search for “a”
Cancel — abort stale requests

Scenario	Before	After
Typing “SYSC4907”	8 requests	1 request
Same search twice	2 requests	0 (cached)

Work in Progress: Electives

The Annoying Part

Right now if you want to know what Science Electives you can take as a Software Eng student, you have to go to a separate website and dig through PDFs.

Same deal for Complementary Studies, tech electives, etc.

Where We're At

- We detect elective types from your audit
- Currently using placeholder data

What We're Adding

1. Add elective tables to the DB
Which electives count for which programs

2. Get the real data
From Carleton's official lists

3. Suggest electives automatically
Based on your program and year

4. Include them in schedule generation
Pick a Science Elective → see all options

Tech Stack

Layer	Tech	Why
Frontend	React + TypeScript	Components, type safety
Styling	TailwindCSS	Fast to write, looks consistent
Backend	Express + Node.js	Simple, fast, JS everywhere
Database	PostgreSQL	Solid, handles complex queries
ORM	Prisma	Type-safe DB queries, easy migrations

How It Works

From upload to registration

1. Upload

Drop in your audit HTML

→

2. Parse

We extract your courses

→

3. Search

Find courses, check prereqs

4. Generate

Click the button

→

5. Pick One

Browse the timetables

→

6. Register

Copy CRNs, paste in Central

Problems We Ran Into

Problem	How We Solved It
Prerequisites have complex AND/OR logic	Store them as trees, walk recursively
Many schedule combinations to check	Backtracking with early exit on conflicts
Search hitting DB too much	Debounce, cache, cancel old requests
Audits aren't all formatted the same	Pattern matching with fallbacks
Page refresh loses your state	URL routing + sessionStorage
Checking time conflicts	Interval overlap check, filter by day first

What's Left (2 Months)

Electives (Main Focus)

- Get official elective lists into the DB
- Link them to programs
- Auto-suggest based on your audit
- Include in schedule generation

UI Polish

- Better way to compare schedules
- Improve visual feedback

Term Selection

- Add ability to pick Fall vs Winter
- Fetch section data for both terms

Testing & Edge Cases

- End-to-end tests
- Handle weird audit formats
- Make sure it doesn't break

Summary

What It Does

✓ Upload your audit once, we remember it

✓ Search any course, see if you qualify

✓ Generate conflict-free schedules

✓ Copy CRNs in one click

How We Built It

- Focused on making it simple to use
- Backtracking to handle the combinatorics
- Clean separation: frontend / backend / DB
- Trees for prerequisite logic
- Debouncing + caching so search is fast

Thanks!

Questions?

Project: SYSC 4907 Capstone
Supervisor: Professor Jerome Talim
Stack: React, Express, PostgreSQL, Prisma, TypeScript